
When and Why Does Hindsight Experience Replay Improve Soft Actor-Critic under Sparse Rewards?

Mustafa Asfari

Mohammad Alkhozami

github.com/mustafa99705/-adrl-sac-her-fetchreach

Abstract

We study when and why Hindsight Experience Replay (HER) helps Soft Actor-Critic (SAC) under sparse rewards, comparing four conditions (dense, dense+HER, sparse, sparse+HER) on a simple reaching task (FetchReach-v3) and a harder pushing task (FetchPush-v3), 5–10 seeds/condition. On the easy task all four conditions solve it near-perfectly; HER only shortens time-to-competence ($2.75\times$ fewer steps to 90% success). On the hard task, plain sparse-reward SAC never learns (5–6% final success, 0 seeds above 90%), while HER on the identical sparse signal reaches 83–91% success — confirming that HER’s benefit grows sharply with task difficulty. Tracking SAC’s auto-tuned entropy coefficient explains why: without HER it collapses toward zero while the policy is still failing (premature convergence), whereas with HER it stays an order of magnitude higher throughout training. A relabeling-ratio ablation ($n_{\text{sampled_goal}} \in \{1, 4, 8\}$) shows this benefit saturates rather than growing indefinitely with more relabeling.

1 Introduction

In real robotic tasks, the most natural reward specification is often binary: the task was either accomplished or it was not. Dense reward functions can provide more feedback, but they usually have to be hand-engineered, require domain knowledge, and may bias the learned behavior in unintended ways. However, standard off-policy actor-critic methods such as Soft Actor-Critic (SAC) can struggle under sparse rewards, because the replay buffer may contain many uninformative failed transitions before the agent observes its first success. While Hindsight Experience Replay (HER) has been shown to improve learning in sparse-reward settings, it is less clear under which task conditions its benefit is most significant, and through which mechanism it acts. This report investigates not only *whether* HER improves SAC, but *when* its contribution becomes meaningful as task difficulty increases, and *why* it helps, by analyzing the training dynamics it induces.

2 Related Work

This project connects three main topics from the lecture and the RL literature: off-policy maximum-entropy reinforcement learning, goal-conditioned reinforcement learning, and reward design under sparse feedback. SAC (1) is the off-policy actor-critic algorithm used as the learning backbone, while HER (2) provides the goal-relabeling mechanism that allows failed trajectories to be reused with achieved goals. The project also relates to Universal Value Function Approximators (4) and multi-goal RL, because both the policy and the value function must condition on the desired goal. The Fetch environments and their sparse/dense reward variants are based on the multi-goal robotics benchmark introduced by Plappert et al. (3) and are available through Gymnasium-Robotics.

3 Contribution and Attribution

We build on existing, unmodified implementations rather than reimplementing published algorithms: SAC and its `HerReplayBuffer` are Stable-Baselines3’s (6), the Fetch environments and their sparse/dense reward variants are Gymnasium-Robotics’ (following Plappert et al. (3)), and our hyperparameters are the tuned Fetch settings from RL Baselines3 Zoo (5) — we did not run our own hyperparameter search. Reimplementing SAC/HER from scratch was explicitly out of scope for this project.

Our own contribution is the experimental design and everything needed to run, diagnose, and interpret it: (1) identifying and fixing a reset bug in `gymnasium-robotics 1.3.x` that silently made the task harder than documented (`env_fix.py`, Section 5); (2) the training/evaluation pipeline (`train.py`) supporting both tasks, all four conditions, and the relabeling-ratio ablation under one shared configuration; (3) `HerDiagnosticsCallback`, which logs the entropy-coefficient and goal-distance-percentile trajectories used for the mechanism analysis in Section 6.2 — this diagnostic is not part of Stable-Baselines3 and was written for this project; (4) designing and running all 70 training runs, including the pilot used to calibrate the FetchPush step budget, and the analysis/plotting pipeline (`plot_results.py`); and (5) the interpretation in Section 6, including the entropy-collapse account of why HER helps and the finding that relabeling-ratio benefit saturates.

4 Approach

We compare four conditions — dense SAC, dense SAC+HER, sparse SAC, and sparse SAC+HER — on `FetchReach-v3`, a relatively simple reaching task, and on `FetchPush-v3`, where the robot must push an object to a target location and sparse-reward learning is known to be substantially harder (2). Hyperparameters are identical across task, reward type, and HER on/off, so that the reward signal / relabeling strategy is the only experimental variable within each comparison. We tested three hypotheses stated in the project proposal: (i) on the simple reaching task, HER provides only a modest speedup, since random exploration occasionally reaches the goal on its own; (ii) on the harder pushing task, HER is expected to substantially improve learning performance; (iii) HER’s effect is visible in the training dynamics, e.g. in the evolution of SAC’s entropy coefficient and in the distribution of relabeled goals, forming an implicit curriculum. Section 6 reports evidence for all three — and, for (ii), an effect considerably larger than "substantial": HER turned out to be the difference between the task being learned at all.

Algorithm 1 SAC with optional Hindsight Experience Replay

Require: environment e , SAC algorithm A , reward function r , HER flag h , relabels per transition k
Ensure: trained goal-conditioned policy $\pi(a \mid s, g)$
Initialize replay buffer R , SAC actor and critic networks
while training budget not reached **do**
 Sample goal g and initial state s_0 from e
 Roll out one episode $(s_0, a_0, \dots, s_T)$ with $\pi(\cdot \mid s, g)$
 Store $(s_t, a_t, r(s_t, a_t, g), s_{t+1}, g)$ in R for all t
 if h is true **then**
 for each t and each of k relabels **do**
 sample $\tau \sim \mathcal{U}(t + 1, T)$; $g' \leftarrow$ achieved goal in s_τ
 store $(s_t, a_t, r(s_t, a_t, g'), s_{t+1}, g')$ in R ▷ often a success: $r = 0$
 end for
 end if
 Update SAC actor and critics on minibatches from R
end while
return π

5 Experimental Setup

Both tasks use Stable-Baselines3 SAC (6) with a `MultiInputPolicy` for the dictionary observations, and share one fixed hyperparameter configuration adopted from the tuned RL Baselines3 Zoo settings

for Fetch-style tasks (5) ($\gamma = 0.95$, learning rate 10^{-3} , batch size 256, replay buffer size 10^6 , HER $n_{\text{sampled_goal}} = 4$ with the *future* strategy where applicable; all remaining settings are Stable-Baselines3 defaults (6)). We deliberately did not tune hyperparameters per condition, to keep the reward signal / HER as the only variable. The success criterion is the environment’s own (end effector, respectively object, within 5 cm of the goal at episode end).

We found and fixed a reset bug in `gymnasium-robotics 1.3.x` (the only release line exposing the -v3 Fetch environments): the initial arm pose was not restored on reset, making the task $\sim 2.5\times$ harder than documented. The fix (`env_fix.py`) backports the official upstream correction and is applied identically to every run below.

Scope actually run. The proposal specified 5 seeds per condition and an ablation over $n_{\text{sampled_goal}} \in \{1, 4\}$. We ran a larger design instead: 10 seeds per condition throughout, and a third ablation setting $n_{\text{sampled_goal}} = 8$, to get tighter confidence bands on FetchPush’s noisier seeds (Section 6 shows this variance was substantial and worth the extra runs). Concretely: on FetchReach-v3 we ran all four conditions with 10 seeds each (100,000 steps/run, evaluating 20 held-out episodes every 2,000 steps). On FetchPush-v3 we ran the two central conditions, sparse and sparse+HER, with 10 seeds each. A short pilot (1 seed/condition, 200,000 steps) showed FetchPush success was still near zero at that budget (0% sparse, 5% sparse+HER) and measured a throughput of 54–61 steps/s; we used this to set the main-grid budget to 10^6 steps/run (matching the proposal’s estimate) and evaluated every 10,000 steps rather than the 2,000 used for Reach, to keep evaluation overhead proportionate to the $10\times$ longer budget. For the ablation, we ran $n_{\text{sampled_goal}} \in \{1, 4, 8\}$ on FetchPush-v3 with 5 seeds per setting ($n_{\text{sampled_goal}} = 4$ reuses the first 5 seeds of the main sparse+HER condition rather than re-running it). In total the study comprises 70 training runs, all completed successfully. To probe *why* HER helps, every run additionally logged SAC’s auto-tuned entropy coefficient α and the distribution of $\|\text{achieved goal} - \text{desired goal}\|$ in sampled replay-buffer minibatches, both every 2,000 (Reach) or 10,000 (Push) steps.

6 Results

6.1 Main comparison: does HER’s benefit grow with task difficulty?

Table 1: As specified in the proposal: 5 seeds/condition.

Task	Condition	Final success	Seeds $\geq 90\%$	Median steps to 90%	AUC
Reach	dense	100.0%	5/5	6,000	0.944
Reach	dense + HER	100.0%	5/5	6,000	0.945
Reach	sparse	98.8%	5/5	22,000	0.804
Reach	sparse + HER	100.0%	5/5	8,000	0.931
Push	sparse	6.2%	0/5	— (never)	0.065
Push	sparse + HER	82.8%	4/5	380,000	0.553

At the protocol’s originally-specified scale, the central finding is already unambiguous: HER is a mild accelerant on Reach and the difference between learning and not learning on Push. Tables 2 and 3 below extend every condition to 10 seeds — purely to tighten the confidence bands on Push’s noisier seeds, discussed in Section 6.2 — and show the conclusion is unchanged, not seed-count-dependent.

Table 2: Extended robustness check: same runs, 10 seeds/condition (FetchReach-v3). All conditions solve the task; HER mainly affects how fast.

Condition	Final success	Seeds $\geq 90\%$	Median steps to 90%	AUC
dense	100.0%	10/10	6,000	0.947
dense + HER	100.0%	10/10	6,000	0.945
sparse	99.4%	10/10	22,000	0.826
sparse + HER	100.0%	10/10	8,000	0.929

Table 2 confirms hypothesis (i): on the easy task, every condition eventually succeeds, and HER’s contribution is a $\sim 2.75\times$ reduction in steps-to-90% for the sparse reward (22,000 $\rightarrow$ 8,000), not

Table 3: Extended robustness check: same runs, 10 seeds/condition (FetchPush-v3). Sparse SAC essentially fails to learn; HER recovers most of the task.

Condition	Final success	Seeds $\geq 90\%$	Median steps to 90%	AUC
sparse	5.4%	0/10	— (never)	0.067
sparse + HER	90.7%	9/10	350,000	0.610

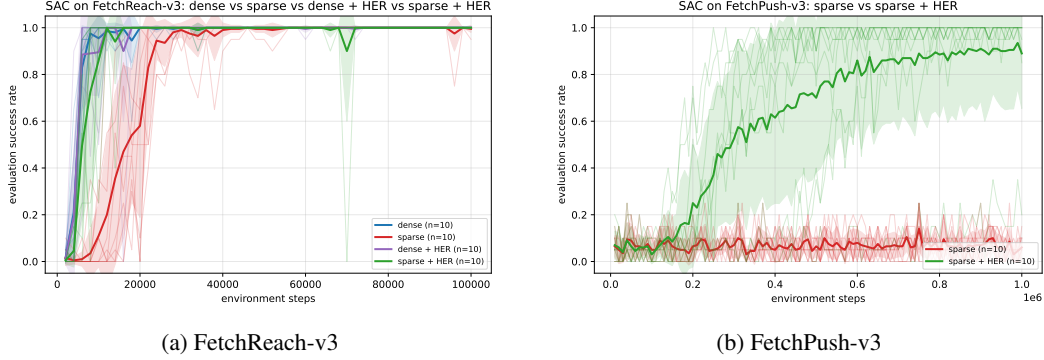


Figure 1: Evaluation success rate vs. environment steps (thin lines: individual seeds; thick line: mean; band: seed range). On Reach, all HER and dense conditions overlap and converge fast, with plain sparse trailing but still reaching near-perfect success. On Push, sparse SAC (red) never rises above noise around 5–10%, while sparse+HER (green) climbs steadily to $\sim 90\%$.

the difference between success and failure. Table 3 and Figure 1b confirm hypothesis (ii) sharply: without HER, SAC extracts essentially nothing from the sparse pushing reward over a full million steps (5.4% final success, statistically indistinguishable from the pre-training baseline), while HER on the identical reward signal reaches 90.7% success. HER’s benefit is not a speedup here — it is the difference between the task being learned at all.

6.2 Mechanism: why does HER help?

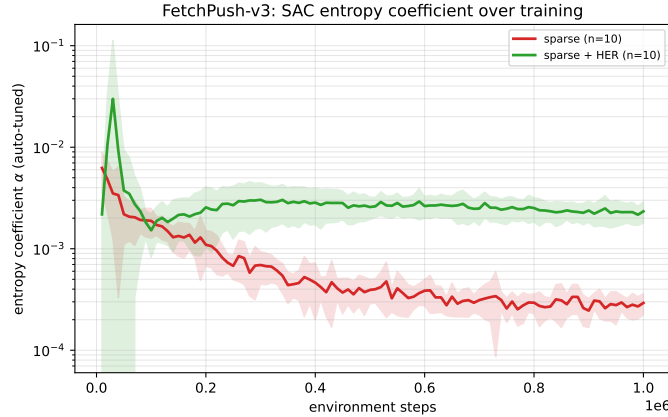


Figure 2: SAC’s auto-tuned entropy coefficient α on FetchPush-v3 (log scale, mean $\pm$ range over 10 seeds). Without HER, α collapses $\sim 21\times$ from 6.2×10^{-3} to 2.9×10^{-4} while the policy is still failing. With HER, α stays roughly flat around $2.2\text{--}2.3 \times 10^{-3}$ throughout, about $8\times$ higher than sparse’s final value.

Figure 2 gives a direct mechanistic explanation for hypothesis (ii): SAC’s entropy term is auto-tuned to keep the policy just exploratory enough to keep improving. Under plain sparse reward on Push, the critic never sees a rewarding transition to correct its estimates, so SAC settles into an overconfident,

low-entropy policy despite *not* having solved the task — a premature-convergence failure mode. With HER supplying manufactured successes from the start, the critic has a genuine learning signal, and α is kept an order of magnitude higher for the full training run, sustaining the exploration needed to eventually discover successful pushes. Notably, entropy collapse *per se* is not the problem: on Reach, α collapses similarly for every condition (from ≈ 0.368 to $\approx 3.1\text{--}3.2 \times 10^{-4}$), but there it coincides with the task actually being solved — collapse is healthy when it tracks genuine competence, and unhealthy when, as in Push without HER, it happens instead of it.

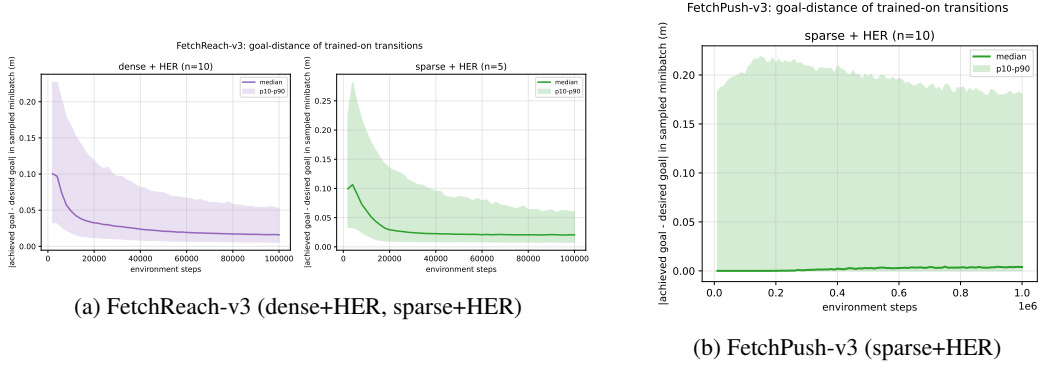


Figure 3: Median (line) and p10–p90 band of $\|\text{achieved} - \text{desired goal}\|$ across transitions actually sampled for training.

For the implicit-curriculum part of hypothesis (iii), the picture differs by task. On Reach (Figure 3a), the median trained-on goal distance falls cleanly as training progresses — from ≈ 0.10 m to $\approx 0.016\text{--}0.020$ m for dense+HER and sparse+HER respectively — a direct, visible curriculum: as the policy improves, the goals it is actually trained on get closer on average. On Push (Figure 3b) the same signal is present but far less visible in this aggregate view: with $n_{\text{sampled_goal}} = 4$, roughly 4 of every 5 stored transitions are HER-reabeled (distance ≈ 0), which pins the median near zero for the entire run and hides the curriculum in the remaining $\sim 20\%$ of real-goal transitions. Those real-goal transitions do show a mild version of the same effect — the p10–p90 band widens from ≈ 0.18 m to a peak of $\approx 0.21\text{--}0.215$ m by $\sim 150\text{--}250$ k steps as the policy starts reaching more diverse states, then narrows back to ≈ 0.18 m by 1M steps — but it is a second-order effect on Push, not the dominant pattern visible in the raw metric. We report this as an honest limitation of the metric rather than evidence against hypothesis (iii): the entropy-coefficient signal (Figure 2) is the clearer mechanism indicator on the harder task.

6.3 Ablation: does more relabeling always help?

The proposal specifies this ablation over $n_{\text{sampled_goal}} \in \{1, 4\}$, 5 seeds/setting; we additionally ran $n_{\text{sampled_goal}} = 8$ at the same 5 seeds/setting to check whether the trend continues past the specified range.

Table 4: HER relabeling-ratio ablation, FetchPush-v3 sparse+HER, 5 seeds/setting (rows 1–4 are the specified range; row 8 is the extension).

$n_{\text{sampled_goal}}$	Final success	Seeds $\geq 90\%$	Median steps to 90%	AUC
1	84.0%	4/5	400,000	0.566
4	82.8%	4/5	380,000	0.553
8	88.6%	4/5	260,000	0.637

Table 4 and Figure 4 address the required ablation: relabeling ratio does not help monotonically or dramatically. $n_{\text{sampled_goal}} = 8$ learns somewhat faster (median steps-to-90% 260,000 vs. $\sim 380\text{--}400,000$) and reaches a marginally higher final success and AUC, but $n_{\text{sampled_goal}} = 1$ and 4 are statistically indistinguishable from each other, and all three settings converge to a similar success region by 1M steps. Every setting also has exactly one seed out of five that fails to reach 90% (final success 0.20–0.45 for that seed, vs. ≥ 0.96 for the other four in the same group; visible as the wide

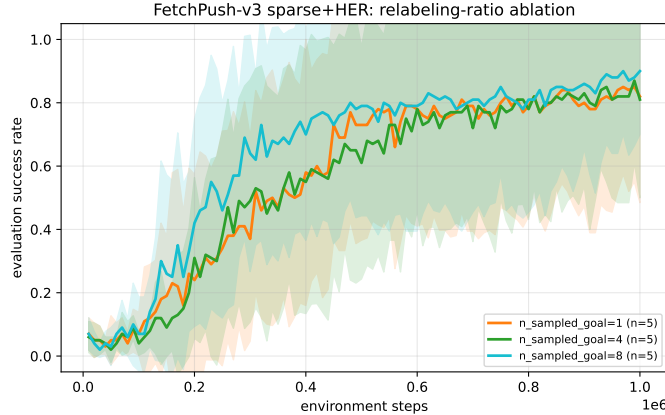


Figure 4: Success rate by HER relabeling ratio on FetchPush-v3 (mean over 5 seeds, band: p10–p90).

lower band in Figure 4) — suggesting per-seed optimization instability on this harder task is a bigger source of variance than the relabeling ratio itself. We conclude the benefit of more relabeling saturates quickly on this task rather than scaling with k .

7 Discussion and Limitations

The central result — HER’s benefit growing from a modest speedup on FetchReach to the difference between learning and not learning on FetchPush — matches all three hypotheses from the proposal, and the entropy-coefficient trajectories (Section 6.2) give a concrete mechanistic account of *why*: HER prevents the premature, overconfident entropy collapse that plain sparse-reward SAC falls into when it never observes a success. We note three limitations. First, both FetchPush conditions show occasional catastrophic-failure seeds (final success as low as 0.20 despite HER); with only 10 (main grid) or 5 (ablation) seeds per setting, these outliers noticeably affect the mean and should be reported alongside the median in any follow-up. Second, the goal-distance curriculum signal, clear on Reach, is largely masked on Push by the relabeling ratio itself (Section 6.2); a cleaner mechanism metric for harder tasks would track only the non-relabeled ("real-goal") transitions directly rather than the pooled distribution. Third, we deviated from the proposal’s literal evaluation cadence (every 2,000 steps) on Push, using 10,000 steps instead to keep evaluation overhead proportionate to the $10\times$ longer training budget; this does not affect the reported metrics’ correctness but is a departure from the originally specified protocol worth flagging explicitly.

8 Reproducibility

All code, per-run logs/metrics/configs, and this report’s source are in the project repository (see title page link); trained model checkpoints (excluded from the repository as large binaries, $\sim 540\text{MB}$ across 70 runs) are available on request. `README.md` documents the software stack (pinned for the `gymnasium-robotics 1.3.x / -v3` environments) and exact commands to reproduce every run; `EXPERIMENT_LOG.md` records the full chronological history of decisions, calibration numbers, and cluster job IDs behind the numbers in this report. `plot_results.py` regenerates all figures and summary tables directly from the tracked per-run data. All 70 training runs (plus a 2-run pilot) were executed on the GWDG/KISSKI HPC cluster, one NVIDIA A100 GPU per run, 4 CPU cores, 16GB RAM; total compute used was ≈ 115 GPU-hours.

9 Conclusion

This report set out to answer not just *whether* HER helps SAC under sparse rewards, but *when* and *why* — both questions are answered directly by the 70 runs above. *When*: HER’s contribution to SAC under sparse rewards is not fixed, it scales with task difficulty — a modest accelerant on an easy task, the difference between success and failure on a harder one. *Why*: this shift is traceable

to HER sustaining healthy exploration (via SAC’s entropy coefficient) where plain sparse rewards let the policy converge prematurely on a failing behavior. More hindsight relabeling helps up to a point but saturates quickly, making $n_{\text{sampled_goal}} = 4\text{--}8$ a reasonable default rather than a setting worth aggressively tuning further.

Natural next steps: (i) re-run the mechanism analysis tracking only non-relabeled transitions to get a cleaner curriculum signal on Push (Section 6.2); (ii) test whether the difficulty-scaling pattern found here (Reach vs. Push) extends to a third, even harder Fetch task (e.g. `FetchSlide-v3` or `FetchPickAndPlace-v3`); and (iii) a formal significance test (e.g. bootstrap confidence intervals) on the seed-level results, given the catastrophic-failure-seed variance noted in Section 7.

References

- [1] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*, 2018.
- [2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, W. Zaremba. Hindsight Experience Replay. *NeurIPS*, 2017.
- [3] M. Plappert, M. Andrychowicz, A. Ray, B. McGrew, B. Baker, G. Powell, J. Schneider, J. Tobin, M. Chociej, P. Welinder, V. Kumar, W. Zaremba. Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research. *arXiv:1802.09464*, 2018.
- [4] T. Schaul, D. Horgan, K. Gregor, D. Silver. Universal Value Function Approximators. *ICML*, 2015.
- [5] A. Raffin. RL Baselines3 Zoo: A Training Framework for Stable-Baselines3 Reinforcement Learning Agents. <https://github.com/DLR-RM/r1-baselines3-zoo>, 2020.
- [6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268), 2021.

Checklist

This checklist is not mandatory, but can help you set up your project in a reproducible manner.

1. General points:
 - (a) Do the main claims made in the abstract and introduction accurately reflect your contributions and scope? [\[Yes\]](#)
 - (b) Did you cite all relevant related work? [\[Yes\]](#)
 - (c) Did you describe the limitations of your work? [\[Yes\]](#) [Section 7](#)
 - (d) Did you include a discussion of future work? [\[Yes\]](#) [Section 6 end / Conclusion](#)
2. If you are including theoretical results...
 - (a) Did you state the full set of assumptions of all theoretical results? [\[NA\]](#) [purely empirical study, no theoretical claims]
 - (b) Did you include complete proofs of all theoretical results? [\[NA\]](#)
3. If you ran experiments (e.g. for benchmarks)...
 - (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [\[Yes\]](#) [GitHub repository linked on the title page; README.md and EXPERIMENT_LOG.md document exact commands and decisions]
 - (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [\[Yes\]](#) [Section 5; hyperparameters taken from RL Baselines3 Zoo, no search performed – Section 3]
 - (c) Did you run at least 10 repetitions of your method? [\[Yes\]](#) [10 seeds/condition for both main comparisons; the ablation uses 5 seeds/setting, as specified in the proposal]
 - (d) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [\[Yes\]](#) [figures show per-seed traces and mean $\pm$ range/p10–p90 bands; per-seed outlier variance discussed explicitly in Section 7]
 - (e) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [\[Yes\]](#) [Section 8: ≈ 115 GPU-hours, $1 \times$ NVIDIA A100/run, GWDG/KISSKI HPC cluster]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
 - (a) If your work uses existing assets, did you cite the creators? [\[Yes\]](#) [Section 3: Stable-Baselines3, Gymnasium-Robotics, RL Baselines3 Zoo]
 - (b) Did you make sure the license of the assets permits usage? [\[Yes\]](#) [Stable-Baselines3 and Gymnasium-Robotics are both MIT-licensed; standard academic/educational use]
 - (c) Did you reference the assets directly within your code and repository? [\[Yes\]](#) [exact versions pinned in `requirements.txt`; imports visible in `train.py`]